

SAVCNG ExcelDNA - Documentación Técnica Completa

Versión: 2.0.0.0

Última Actualización: 9 de enero de 2025

Estado: En Desarrollo

Tabla de Contenidos

1. [Descripción General](#)
 2. [Arquitectura del Proyecto](#)
 3. [Componentes Principales](#)
 4. [Sistema de Validaciones Detallado](#)
 5. [FrmValidaciones.cs - Análisis Profundo](#)
 6. [Flujo de Trabajo](#)
 7. [Especificaciones Técnicas](#)
 8. [Guía de Desarrollo](#)
 9. [Patrones de Código y Mejores Prácticas](#)
 10. [Troubleshooting y Casos Especiales](#)
-

Descripción General

SAVCNG ExcelDNA es un complemento (Add-in) de Microsoft Excel desarrollado en C# (.NET Framework 4.8) que proporciona un sistema avanzado de validación de datos para archivos de censo de gobierno.

El proyecto utiliza **ExcelDNA** como framework de integración con Excel, permitiendo crear un menú personalizado en la cinta de opciones (Ribbon) de Excel desde el cual se pueden cargar archivos de censo y aplicar múltiples validaciones de integridad de datos.

Propósito Principal

- Garantizar la calidad e integridad de los datos en archivos de censo
- Proporcionar una interfaz gráfica intuitiva para validación de datos
- Detectar y resaltar errores de formato y contenido
- Facilitar listas desplegables (Data Validation) personalizadas
- Validar presencia de respuestas "No Aplica" (NS) con alertas contextuales

Tecnologías Utilizadas

- **Lenguaje:** C# 7.3
- **Framework:** .NET Framework 4.8
- **Librería de Interop:** Microsoft.Office.Interop.Excel
- **Plugin Framework:** ExcelDNA
- **UI:** Windows Forms (.NET Framework)

Arquitectura del Proyecto

```
SAVCNG_ExcelDNA/
├─ MenuCenso.cs           (Punto de entrada y menú)
├─ FrmValidaciones.cs     (Lógica principal de validaciones)
├─ FrmValidaciones.Designer.cs (Diseño de formulario)
├─ Properties/
│   └─ AssemblyInfo.cs    (Metadatos del ensamblado)
├─ bin/                   (Ejecutables compilados)
├─ obj/                   (Artefactos compilados)
└─ documentacion/
    └─ documentacion_savcng_YYYY-MM-DD.md (Documentación técnica)
```

Patrón Arquitectónico

- **Model:** Las colecciones (List, Dictionary) almacenan datos de validación y rangos
- **View:** Windows Forms (FrmValidaciones) para la interfaz de usuario
- **Controller:** Métodos de eventos que procesan interacciones del usuario
- **Service:** Métodos estáticos y funciones auxiliares para lógica compartida

Componentes Principales

1. MenuCenso.cs - Punto de Entrada

Responsable de la integración con Excel y la interfaz de usuario del ribbon.

Características Principales:

```
public class MenuCenso : ExcelRibbon
{
    // Define un menú personalizado en la cinta de Excel
    public override string GetCustomUI(string RibbonID)
}
```

Funcionalidades:

- **XML Ribbon:** Define la estructura del menú en la pestaña "SAVCNG - Censo"
- **Diálogo de Archivo:** Permite seleccionar archivos de censo (.xlsx, .xlsm, .xlsb)
- **Gestión de Workbook:** Abre el archivo y lo pasa a la ventana de validaciones
- **Manejo de Errores:** Captura excepciones durante la carga de archivos

Flujo de Ejecución:

1. Usuario hace clic en "Cargar Censo" en el menú de Excel
2. Se abre un diálogo `OpenFileDialog`
3. El archivo seleccionado se abre en Excel mediante `excelApp.Workbooks.Open()`
4. Se instancia `FrmValidaciones` pasando el workbook
5. La ventana se muestra flotante sobre Excel

2. `FrmValidaciones.cs` - Motor de Validaciones

Este es el corazón del aplicativo. Implementa todas las validaciones de datos y maneja la interacción del usuario.

2.1 Variables de Estado Principales

```
private Excel.Workbook _libroCenso;
```

- **Propósito:** Referencia al archivo de Excel cargado
- **Tipo:** `Excel.Workbook` (objeto COM de Office)
- **Inicialización:** En el constructor, recibida desde `MenuCenso.cs`
- **Ciclo de vida:** Se mantiene mientras la ventana esté abierta
- **Notas:** Es un objeto COM, requiere manejo cuidadoso de memoria

```
private Excel.Range _rangoCapturado;
```

- **Propósito:** Almacena el rango de celdas seleccionado por el usuario
- **Tipo:** `Excel.Range` (objeto COM de Office)
- **Inicialización:** Se asigna en `btnCapturarRango_Click()`
- **Validación:** Se verifica que sea un rango (no imagen, gráfico, etc.)
- **Uso:** Base para aplicar todas las validaciones
- **Notas:** Puede contener múltiples áreas (rangos no contiguos)

```
private Excel.Range _rangoCatalogo;
```

- **Propósito:** Almacena el rango que contiene las opciones para listas desplegables
- **Tipo:** `Excel.Range`
- **Inicialización:** Se asigna mediante `InputBox` en validación de catálogos
- **Estado Actual:** Declarado pero actualmente no utilizado en lógica activa
- **Potencial:** Podría usarse para caché de opciones de catálogos

```
private System.Collections.Generic.List<Excel.Range> _pendientesCatalogo;
```

- **Propósito:** Colecciona rangos que requieren validación de catálogo
- **Tipo:** `List<Excel.Range>`
- **Inicialización:** Nueva lista en declaración
- **Estado Actual:** Actualmente NO se utiliza en la implementación actual
- **Potencial Futuro:** Para procesamiento por lotes de validaciones

```
private System.Collections.Generic.Dictionary<string, string> _preguntaPorRango;
```

- **Propósito:** Mapea direcciones de rango a números de pregunta
- **Tipo:** `Dictionary<string, string>` (clave: dirección, valor: número)
- **Inicialización:** Nueva lista en declaración
- **Estado Actual:** Actualmente NO se utiliza en la implementación actual
- **Potencial Futuro:** Para historial y auditoría de validaciones

2.2 Constructor

```
public FrmValidaciones(Excel.Workbook libroAbierto)
{
    InitializeComponent();           // Genera los controles del formulario
    _libroCenso = libroAbierto;      // Almacena referencia al workbook
    lblCenso.Text = "Censo cargado: " + _libroCenso.Name; // UI update
    this.TopMost = true;             // Mantener siempre visible
    this.StartPosition = FormStartPosition.CenterScreen; // Centrar en pantalla
}
```

Detalles de Implementación:

- `InitializeComponent()`: Método generado automáticamente por Visual Studio
- `_libroCenso.Name`: Extrae el nombre del archivo (ej. "censo_2024.xlsx")
- `TopMost = true`: Crucial para que la ventana no quede oculta bajo Excel
- `StartPosition.CenterScreen`: Mejora UX mostrando la ventana en posición visible

Sistema de Validaciones Detallado

3.1 Validación de Decimales (Enteros)

Objetivo: Detectar y resaltar celdas que contienen números con decimales cuando solo se permiten números enteros.

Implementación Técnica:

```

if (chkDecimales.Checked == true)
{
    _rangoCapturado.FormatConditions.Delete(); // Limpiar formatos previos

    Excel.Range primeraCelda = _rangoCapturado.Cells[1, 1];
    string direccion = primeraCelda.get_Address(false, false);

    string formula = $"=Y(ESNUMERO({direccion}), TRUNCAR({direccion})<>
{direccion})";

    Excel.FormatCondition formato = (Excel.FormatCondition)
        _rangoCapturado.FormatConditions.Add(
            Excel.XlFormatConditionType.xlExpression,
            Type.Missing,
            formula);

    formato.Interior.Color =
        System.Drawing.ColorTranslator.ToOle(System.Drawing.Color.Yellow);
    formato.Font.Color =
        System.Drawing.ColorTranslator.ToOle(System.Drawing.Color.Red);
    formato.Font.Bold = true;
}

```

Lógica de la Fórmula:

- **ESNUMERO(A1)**: Verifica que el valor sea numérico
- **TRUNCAR(A1)**: Extrae la parte entera (sin decimales)
- **TRUNCAR(A1) <> A1**: Si la parte entera difiere del original, hay decimales
- **Y(...)**: Todas las condiciones deben cumplirse

Análisis Paso a Paso:

1. Borra cualquier formato condicional anterior para evitar conflictos
2. Obtiene la dirección relativa de la primera celda (ej. "A1")
3. Construye fórmula en idioma Excel (ESNUMERO, TRUNCAR son localizadas)
4. Crea **FormatCondition** con tipo **xlExpression** (evaluación de fórmula)
5. Aplica estilos visuales: fondo amarillo, texto rojo, negrita
6. Desmarca el checkbox para preparar siguiente operación

Casos Especiales:

- **Celdas vacías**: **ESNUMERO()** retorna FALSE, no se resaltan
- **Texto**: **ESNUMERO()** retorna FALSE, no se resaltan
- **Números sin decimales**: Fórmula retorna FALSE, no se resaltan
- **Números negativos con decimales**: Se resaltan correctamente

Limitaciones Conocidas:

- Solo detecta primera celda del rango para la dirección
 - Si el rango tiene múltiples áreas, solo valida en base a la primera
-

3.2 Validación de Catálogos (Listas Desplegables)

Objetivo: Crear listas desplegables (Data Validation) con opciones predefinidas.

Flujo de Trabajo:

1. Usuario selecciona rango de datos
↓
2. Presiona "Capturar Rango"
↓
3. Marca checkbox "Catálogos"
↓
4. Presiona "Aplicar"
↓
5. InputBox pide seleccionar rango de opciones
↓
6. Usuario selecciona rango de catálogo
↓
7. Se crea lista desplegable en el rango original

Implementación Técnica:

```
else if (chkCatalogos.Checked == true)
{
    // Solicitar rango de opciones al usuario
    object resultadoInput = excelApp.InputBox(
        "Selecciona con el ratón el rango que contiene las opciones...",
        "Seleccionar Opciones del Catálogo",
        Type.Missing, Type.Missing, Type.Missing, Type.Missing, Type.Missing, 8);

    // Verificar cancelación
    if (resultadoInput is bool && (bool)resultadoInput == false)
    {
        chkCatalogos.Checked = false;
        return;
    }

    // Convertir selección a rango de Excel
    Excel.Range rangoOpciones = (Excel.Range)resultadoInput;

    // Obtener dirección con nombre de hoja (ej. '=Sheet1!$A$1:$A$5')
```

```

string formulaOpciones = "=" + rangoOpciones.get_Address(true, true,
    Excel.XlReferenceStyle.xlA1, true);

// Aplicar validación
_rangoCapturado.Validation.Delete();

_rangoCapturado.Validation.Add(
    Excel.XlDVType.xlValidateList,
    Excel.XlDValertStyle.xlValidAlertStop,
    Excel.XlFormatConditionOperator.xlBetween,
    formulaOpciones,
    Type.Missing);

_rangoCapturado.Validation.InCellDropdown = true;
_rangoCapturado.Validation.IgnoreBlank = true;
}

```

Detalles Técnicos de Parámetros:

- **XlDVType.xlValidateList**: Tipo de validación (lista)
- **XlDValertStyle.xlValidAlertStop**: Bloquea entradas inválidas
- **XlFormatConditionOperator.xlBetween**: Operador (requerido pero no usado en listas)
- **formulaOpciones**: Referencia con nombre de hoja y referencias absolutas
- **InCellDropdown = true**: Muestra la flechita desplegable
- **IgnoreBlank = true**: Permite celdas vacías inicialmente

Parámetro **Type.Missing**:

- En Interop COM, representa valor no aplicable
- Equivalente a **omitted** en VBA
- Necesario para evitar errores en parámetros opcionales

InputDialog con Tipo 8:

- El último parámetro **8** es crucial
- Indica a Excel que espera selección de rango con el ratón
- Retorna un **Excel.Range** si selecciona, **false** si cancela
- Mejora UX al permitir seleccionar visualmente

Construcción de Dirección:

```

string formulaOpciones = "=" + rangoOpciones.get_Address(true, true,
    Excel.XlReferenceStyle.xlA1, true);

// Ejemplo resultado: "='Hoja2'!$A$1:$A$10"

```

- `true`, `true`: Referencias absolutas (\$A\$1)
- `x1A1`: Formato A1 (no R1C1)
- `true` final: Incluir nombre de hoja

3.3 Validación NS (No Aplica) - Caso Especializado

Objetivo: Permitir números ≥ 0 o valor "NS", con alerta automática cuando se usa NS.

Contexto: En censos, "NS" significa "No Sabe" o "No Aplica" y requiere justificación.

Implementación Técnica:

```
else if (chkNS.Checked == true)
{
    _rangoCapturado.Validation.Delete();

    // Obtener dirección sin caracteres especiales
    Excel.Range primeraCelda = (Excel.Range)_rangoCapturado.Cells[1, 1];
    string direccion = primeraCelda.Address.Replace("$", "");

    // Fórmula: números  $\geq 0$  o el valor "NS"
    string formulaRestriccion =
        $"=O(Y(ESNUMERO({direccion}){separador}{direccion}>=0){separador}
{direccion}=\"NS\")";

    // Aplicar validación
    _rangoCapturado.Validation.Add(
        Excel.XlDVType.xlValidateCustom,
        Excel.XlDValAlertStyle.xlValidAlertStop,
        Excel.XlFormatConditionOperator.xlBetween,
        formulaRestriccion,
        Type.Missing);

    _rangoCapturado.Validation.IgnoreBlank = true;
    _rangoCapturado.Validation.InCellDropdown = true;
    _rangoCapturado.Validation.ErrorTitle = "Error de validación";
    _rangoCapturado.Validation.ErrorMessage =
        "Solo se permiten números mayores o iguales a cero, o el valor 'NS'.";
    _rangoCapturado.Validation.ShowError = true;

    // --- PARTE 2: Calcular alerta automática ---
    int filaSiguiente = _rangoCapturado.Row + _rangoCapturado.Rows.Count;
    Excel.Worksheet ws = (Excel.Worksheet)_rangoCapturado.Worksheet;
    Excel.Range celdaAlerta = (Excel.Range)ws.Cells[filaSiguiente, "K"];

    // Construir suma de CONTAR.SI para todas las áreas
    System.Collections.Generic.List<string> partesCountIf =
        new System.Collections.Generic.List<string>();
```



```

for (int i = 1; i <= _rangoCapturado.Areas.Count; i++)
{
    Excel.Range area = (Excel.Range)_rangoCapturado.Areas[i];
    string addrAbs = area.Address;
    partesCountIf.Add($"CONTAR.SI({addrAbs}{separador}\"NS\")");
}

string sumaInner = string.Join(separador, partesCountIf);
string textoAlerta = "Alerta: debido a que cuenta con registros NS, " +
    "debe proporcionar una justificación en el área de comentarios...";

string formulaFinalAlerta =
    $"=SI(SUMA({sumaInner})<>0{separador}\"{textoAlerta}\"{separador}\" \")";

celdaAlerta.FormulaLocal = formulaFinalAlerta;

chkNS.Checked = false;
MessageBox.Show("Validación NS y Mensaje de Alerta configurados correctamente.",
    "SAVCNG", MessageBoxButtons.OK, MessageBoxIcon.Information);
}

```

Análisis Detallado:

Parte 1: Validación de Entrada

Fórmula: =O(Y(ESNUMERO(A1),A1>=0),"NS")

- └ Y(ESNUMERO(A1), A1>=0): Es número y >= 0
- └ O(...): 0 es número válido 0 es texto "NS"

Parte 2: Alerta Automática

Cálculo de ubicación:

- filaSiguiente = rangoRow + rangoRows.Count
- Ejemplo: Si rango es A5:A10 (6 filas)
 - filaSiguiente = 5 + 6 = 11
 - Alerta se pone en K11

Fórmula de alerta:

=SI(SUMA(CONTAR.SI(A5:A10;"NS"))<>0;"Alerta: ...;")

- └ CONTAR.SI(A5:A10;"NS"): Cuenta cuántos NS
- └ SUMA(...)<>0: Hay al menos uno
- └ SI(...): Muestra texto si hay NS

Manejo de Rangos No Contiguos:

```
for (int i = 1; i <= _rangoCapturado.Areas.Count; i++)
{
    Excel.Range area = (Excel.Range)_rangoCapturado.Areas[i];
    // Si el usuario selecciona: A1:A5 y C1:C5 (dos áreas)
    // El loop procesa ambas y las suma
}
```

Uso de **FormulaLocal** vs **Formula**:

- **FormulaLocal**: Usa funciones en idioma del sistema (CONTAR.SI, SI)
- **Formula**: Usa funciones en inglés (COUNTIF, IF)
- Crucial para compatibilidad multiidioma

Separador Dinámico:

```
string separador = excelApp.International[
    Excel.XlApplicationInternational.xlListSeparator].ToString();
// En español: ";" (punto y coma)
// En inglés: "," (coma)
```

FrmValidaciones.cs - Análisis Profundo

4.1 Método: btnCapturarRango_Click

Responsabilidad: Capturar el rango seleccionado por el usuario y detectar contexto.

```
private void btnCapturarRango_Click(object sender, EventArgs e)
{
    try
    {
        Microsoft.Office.Interop.Excel.Application excelApp =
            (Microsoft.Office.Interop.Excel.Application)ExcelDna.Integration.ExcelDnaUtil.Applic
            ation;

        object seleccion = excelApp.Selection;

        if (seleccion is Excel.Range)
        {
            _rangoCapturado = (Excel.Range)seleccion;

            string pregunta = ObtenerNumeroPregunta(_rangoCapturado);
            lblPregunta.Text = "Pregunta detectada: " + pregunta;
        }
    }
}
```

```
        MessageBox.Show("Se capturó correctamente el rango: " +
            _rangoCapturado.Address,
            "Captura exitosa", MessageBoxButtons.OK,
            MessageBoxIcon.Information);
    }
    else
    {
        MessageBox.Show("Por favor, selecciona celdas de Excel, " +
            "no imágenes ni gráficos.",
            "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
    }
}
catch (Exception ex)
{
    MessageBox.Show("Ocurrió un error al capturar: " + ex.Message,
        "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
```

Flujo Paso a Paso:

1. Obtener Aplicación Excel:

```
excelApp = (Microsoft.Office.Interop.Excel.Application)
    ExcelDna.Integration.ExcelDnaUtil.Application;
```

- `ExcelDnaUtil.Application` obtiene instancia de Excel activa
- Cast a `Application` para acceder a propiedades

2. Obtener Selección:

```
object seleccion = excelApp.Selection;
```

- Retorna un `object` porque podría ser: Range, Picture, Shape, Chart, etc.
- Por eso es necesario el tipo check con `is`

3. Validar Tipo:

```
if (seleccion is Excel.Range)
```

- Patrón `is` es seguro (tipo check + cast)

- Si no es Range, muestra mensaje informativo

4. Guardar Rango:

```
_rangoCapturado = (Excel.Range)seleccion;
```

- Almacena para uso posterior en validaciones
- Mantiene vivo el objeto COM durante sesión

5. Detectar Pregunta:

```
string pregunta = ObtenerNumeroPregunta(_rangoCapturado);  
lblPregunta.Text = "Pregunta detectada: " + pregunta;
```

- Busca el identificador en columna A hacia arriba
- Proporciona contexto al usuario

6. Feedback al Usuario:

```
MessageBox.Show("Se capturó correctamente el rango: " +  
_rangoCapturado.Address);
```

- Muestra dirección (ej. "A1:C10") para confirmación

Manejo de Errores:

- Try-catch captura excepciones COM
- Muestra mensaje al usuario sin romper la aplicación
- Errores posibles:
 - Excel no está abierto
 - Workbook fue cerrado externamente
 - Permisos de acceso insuficientes

4.2 Método: btnAplicar_Click

Responsabilidad: Orquestrar la aplicación de validaciones según checkboxes marcados.

Este es el método más complejo del formulario. Se divide en tres rutas principales.

Estructura General:

```
private void btnAplicar_Click(object sender, EventArgs e)
{
    Excel.Application excelApp =
    (Excel.Application)ExcelDna.Integration.ExcelDnaUtil.Application;
    string separador =
    excelApp.International[Excel.XlApplicationInternational.xlListSeparator].ToString();

    try
    {
        if (_rangoCapturado == null)
        {
            MessageBox.Show("¡Espera! Primero debes capturar un rango.",
                "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
            return;
        }

        if (chkDecimales.Checked == true)
        {
            // RUTA 1: Validación de Decimales
        }
        else if (chkCatalogos.Checked == true)
        {
            // RUTA 2: Validación de Catálogos
        }
        else if (chkNS.Checked == true)
        {
            // RUTA 3: Validación NS
        }
        else
        {
            MessageBox.Show("No has marcado ninguna validación para aplicar.",
                "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Ocurrió un error al aplicar el formato: " + ex.Message,
            "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Pre-requisitos Validados:

1. `_rangoCapturado != null`: Se requiere haber capturado un rango
2. Exactamente un checkbox marcado: El flujo es de elección única
3. Separador de lista del sistema: Para internacionalización

4.3 Método: ObtenerNumeroPregunta

Responsabilidad: Detectar el identificador de la pregunta del censo.

```
private string ObtenerNumeroPregunta(Excel.Range rango)
{
    try
    {
        Excel.Worksheet hoja = rango.Worksheet;
        int filaInicial = rango.Row;

        // Buscar desde la fila seleccionada hacia arriba en columna A
        for (int f = filaInicial; f >= 1; f--)
        {
            Excel.Range celdaA = hoja.Cells[f, 1];
            object valor = celdaA.Value2;

            if (valor != null && !string.IsNullOrEmpty(valor.ToString().Trim()))
            {
                return valor.ToString().Trim();
            }
        }
    }
    catch { /* Silencio en error */ }

    return "(no encontrada)";
}
```

Lógica Paso a Paso:

1. Obtener Contexto:

```
Excel.Worksheet hoja = rango.Worksheet;
int filaInicial = rango.Row;
```

- **Worksheet:** Hoja donde está el rango
- **Row:** Primera fila del rango

2. Buscar Hacia Arriba:

```
for (int f = filaInicial; f >= 1; f--)
```

- Comienza en la fila del rango

- Desciende hacia arriba (f--)
- Detiene en fila 1

3. Acceder Celda Columna A:

```
Excel.Range celdaA = hoja.Cells[f, 1];
object valor = celdaA.Value2;
```

- `Cells[fila, columna]`: Acceso directo (1-indexed)
- `Value2`: Valor del formulario (más eficiente que `Value`)
- Retorna `object` porque puede ser número, texto, null

4. Validar Valor:

```
if (valor != null && !string.IsNullOrEmpty(valor.ToString().Trim()))
```

- `valor != null`: Celda no está vacía
- `ToString().Trim()`: Convertir a string y quitar espacios
- `!IsNullOrEmpty()`: Verificar que no sea solo espacios

5. Retornar Resultado:

- Si encuentra: Retorna el valor trimmed
- Si no encuentra: Retorna "(no encontrada)"

Caso de Uso:

Estructura típica de censo:

Fila 1: [Título de Pregunta]

Fila 2: [Número: 01]

Fila 3: [Usuario selecciona celdas aquí] ← ObtenerNumeroPregunta inicia aquí

...

Fila N: [Datos]

Resultado: "01" (número de pregunta)

Manejo de Excepciones:

- `catch { }` silencioso: Retorna valor por defecto
- Excepciones esperadas:
 - Worksheet fue eliminada
 - Workbook fue cerrado

- Permisos insuficientes

4.4 Método: chkCatalogos_CheckedChanged

Responsabilidad: Validar precondiciones cuando se marca el checkbox.

```
private void chkCatalogos_CheckedChanged(object sender, EventArgs e)
{
    if (chkCatalogos.Checked)
    {
        if (_libroCenso == null || _rangoCapturado == null)
        {
            MessageBox.Show("Primero carga un censo y captura un rango con el
botón.",
                "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
            chkCatalogos.Checked = false;
        }
    }
}
```

Validaciones:

- `_libroCenso == null`: Significa que no se cargó archivo
- `_rangoCapturado == null`: Significa que no se capturó rango
- Si faltan requisitos: Desmarca el checkbox automáticamente

UX Improvement:

- Previene que usuario marque casilla sin estar listo
- Feedback inmediato sobre qué falta
- Desmarcado automático = estado consistente

4.5 Método: chkNS_CheckedChanged

Responsabilidad: Validar precondiciones para validación NS.

```
private void chkNS_CheckedChanged(object sender, EventArgs e)
{
    if (chkNS.Checked)
    {
        if (_libroCenso == null || _rangoCapturado == null)
        {
            MessageBox.Show("Primero carga un censo y captura un rango con el
botón.",
                "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
        }
    }
}
```



```

        chkNS.Checked = false;
    }
}
}

```

Idéntico a `chkCatalogos_CheckedChanged`:

- Mismo patrón de validación
- Mismo mensaje de error
- Mantiene consistencia en UX

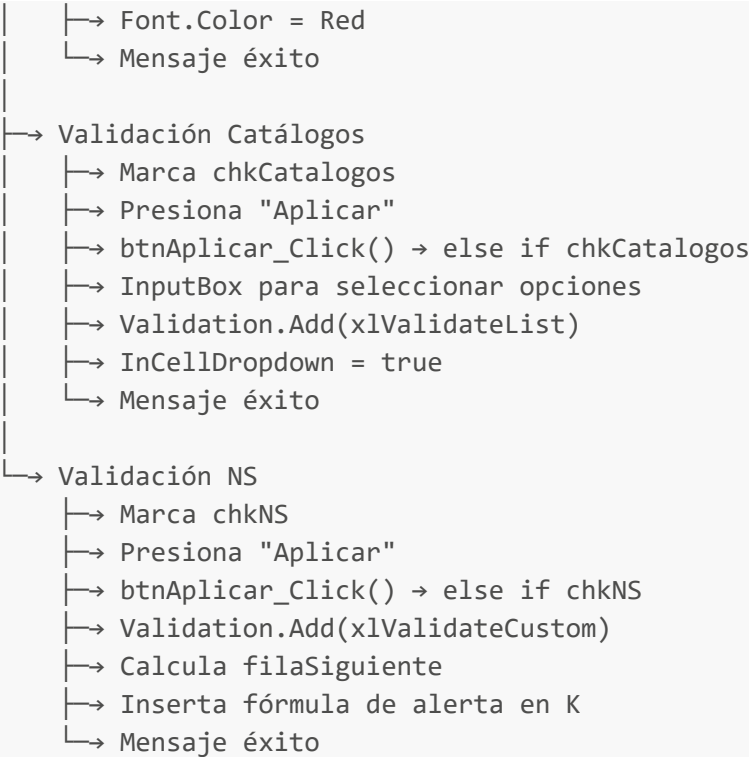
Flujo de Trabajo

Secuencia Completa de Operación

```

INICIO
|
|→ [MenuCenso.cs] Usuario click "Cargar Censo"
|   |→ OpenFileDialog
|   |→ excelApp.Workbooks.Open(archivo)
|   |→ new FrmValidaciones(workbook)
|
|→ [FrmValidaciones.cs] Ventana se abre
|   |→ Constructor:
|   |   |→ Almacena _libroCenso
|   |   |→ Muestra nombre en etiqueta
|   |   |→ TopMost = true (siempre visible)
|   |
|   |→ Usuario selecciona rango en Excel
|   |   |→ Presiona "Capturar Rango"
|   |
|   |→ btnCapturarRango_Click():
|   |   |→ excelApp.Selection
|   |   |→ Validación: is Excel.Range
|   |   |→ Guarda en _rangoCapturado
|   |   |→ ObtenerNumeroPregunta()
|   |   |→ Muestra mensaje éxito
|   |
|   |→ Usuario marca checkbox de validación
|   |   |→ Validación Decimales
|   |   |   |→ Marca chkDecimales
|   |   |   |→ Presiona "Aplicar"
|   |   |   |→ btnAplicar_Click() → if chkDecimales
|   |   |   |→ FormatConditions.Add()
|   |   |   |→ Interior.Color = Yellow

```



Especificaciones Técnicas

Requisitos del Sistema

Aspecto	Requisito
Versión .NET	.NET Framework 4.8
Versión C#	7.3
Versión Excel	2013 o superior
SO	Windows 7/8/10/11 (32 o 64 bits)
Memoria RAM Mínima	4 GB
Espacio en Disco	~100 MB
Bitness	Mismo que Office instalado (32 o 64)

Dependencias Principales

```
<!-- Librería de integración con Excel -->
Microsoft.Office.Interop.Excel (versión 15.0+)

<!-- Framework ExcelDNA -->
ExcelDna.Integration (versión 0.34+)
```

```
ExcelDna.Integration.CustomUI

<!-- .NET Framework incluido -->
System.Windows.Forms
System.Drawing
System.Collections.Generic
```

Internacionalización

Funciones Localizadas:

Español:	Inglés:	Francés:
ESNUMERO()	ISNUMBER()	ISNUMBER()
TRUNCAR()	TRUNCAR()	TRUNCAR()
CONTAR.SI()	COUNTIF()	COUNTIFS()
SI()	IF()	SI()
Y()	AND()	ET()
O()	OR()	OU()
MINUSC()	LOWER()	MINUSCULE()
EXACTO()	EXACT()	EXACT()
ESTEXTO()	ISTEXT()	ISTEXT()

Separador de Argumentos:

```
string separador = excelApp.International[
    Excel.XlApplicationInternational.xlListSeparator].ToString();

// España/Hispanoamérica: ";"
// EUA/Canada: ","
// Francia: ";"
// Italia: ";"
```

Rendimiento

Operación	Tiempo Estimado	Variables
Apertura formulario	< 500 ms	Según carga Excel
Captura de rango	Inmediato	Depende tamaño selección
Validación decimales	1-2 seg	Depende tamaño rango
Validación catálogos	2-3 seg	Depende tamaño rango
Validación NS	3-5 seg	Rangos no contiguos

Operación	Tiempo Estimado	Variables
Memoria usada	50-100 MB	Según archivos abiertos

Límites Conocidos

- **Máximo rango:** 1,048,576 filas × 16,384 columnas (límite Excel)
- **Máximo opciones catálogo:** ~256 opciones (limitación Interop)
- **Máximo formato condicional:** 3 reglas por celda (limitación Excel)
- **Máximo rangos no contiguos:** Sin límite teórico

Guía de Desarrollo

Estructura de Código

Pattern 1: Inicialización Segura de Objetos COM

```
// CORRECTO: Obtener aplicación y validar
Microsoft.Office.Interop.Excel.Application excelApp =
    (Microsoft.Office.Interop.Excel.Application)
    ExcelDna.Integration.ExcelDnaUtil.Application;

// Verificar que se obtuvo correctamente
if (excelApp == null)
{
    throw new InvalidOperationException("Excel no está disponible");
}
```

Pattern 2: Validación de Tipos con 'is'

```
// RECOMENDADO: Usar patrón 'is'
object seleccion = excelApp.Selection;

if (seleccion is Excel.Range)
{
    Excel.Range rango = (Excel.Range)seleccion;
    // Usar rango con seguridad
}

// ALTERNATIVA: Casting con excepción
try
{
    Excel.Range rango = (Excel.Range)seleccion;
}
```

```
catch (InvalidCastException)
{
    MessageBox.Show("No es un rango válido");
}
```

Pattern 3: Cleanup de Objetos COM

```
// En C# con Office Interop, los objetos COM no se limpian automáticamente
// Es importante liberar referencias cuando sea apropiado

// Para objetos temporales
Excel.Range temp = worksheets.Cells[1, 1];
// Usar temp...
System.Runtime.InteropServices.Marshal.ReleaseComObject(temp);

// Para objetos de larga vida (como _libroCenso)
// Se recomienda mantener vivos durante toda la sesión
```

Pattern 4: Manejo de Excepciones COM

```
try
{
    // Operaciones COM
    _rangoCapturado.FormatConditions.Add(...);
}
catch (System.Runtime.InteropServices.COMException comEx)
{
    // Error específico COM (HRESULT 0x800A...)
    MessageBox.Show($"Error COM: {comEx.ErrorCode:X}");
}
catch (Exception ex)
{
    // Error genérico
    MessageBox.Show($"Error: {ex.Message}");
}
```

Extensiones Recomendadas

1. Agregar Nueva Validación

Pasos:

1. Crear checkbox en Designer:

- Abrir `FrmValidaciones.Designer.cs`
- Agregar `CheckBox chkMiValidacion`
- Configurar propiedades básicas

2. Crear método privado:

```
private void AplicarMiValidacion()
{
    try
    {
        // Lógica de validación aquí
        _rangoCapturado.FormatConditions.Add(...);

        MessageBox.Show("Validación aplicada", "Éxito",
            MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error: {ex.Message}", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

3. Agregar en `btnAplicar_Click`:

```
else if (chkMiValidacion.Checked == true)
{
    AplicarMiValidacion();
    chkMiValidacion.Checked = false;
}
```

4. Agregar en `CheckedChanged`:

```
private void chkMiValidacion_CheckedChanged(object sender, EventArgs e)
{
    if (chkMiValidacion.Checked)
    {
        if (_libroCenso == null || _rangoCapturado == null)
        {
            MessageBox.Show("Primero carga un censo y captura un rango.",
                "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
            chkMiValidacion.Checked = false;
        }
    }
}
```

```
}
}
```

2. Mejoras de Internacionalización

Actual: Las fórmulas usan funciones localizadas detectadas en tiempo de ejecución.

Posible Mejora:

```
public class FormulaLocalizador
{
    private Dictionary<string, string> funcionesPorIdioma;

    public FormulaLocalizador(Excel.Application excelApp)
    {
        // Detectar idioma actual
        CultureInfo idioma = new CultureInfo(excelApp.LanguageSettings.LanguageID);

        // Cargar funciones según idioma
        if (idioma.Name.StartsWith("es"))
        {
            funcionesPorIdioma = ObtenerFuncionesEspañol();
        }
        else if (idioma.Name.StartsWith("en"))
        {
            funcionesPorIdioma = ObtenerFuncionesIngles();
        }
    }

    public string ObtenerFuncion(string nombreEstandar)
    {
        return funcionesPorIdioma.ContainsKey(nombreEstandar)
            ? funcionesPorIdioma[nombreEstandar]
            : nombreEstandar;
    }
}
```

3. Sistema de Caché para Opciones de Catálogos

Actual: Cada vez que se selecciona un catálogo, se debe elegir el rango.

Posible Mejora:

```
private Dictionary<string, Excel.Range> cacheRangosCatalogo =
    new Dictionary<string, Excel.Range>();
```

```
private Excel.Range ObtenerRangoCatalogoCacheado(string nombreCatalogo)
{
    if (!cacheRangosCatalogo.ContainsKey(nombreCatalogo))
    {
        // Pedir al usuario
        object resultado = excelApp.InputBox(...);
        cacheRangosCatalogo[nombreCatalogo] = (Excel.Range)resultado;
    }

    return cacheRangosCatalogo[nombreCatalogo];
}
```

4. Reporte de Validaciones Aplicadas

Actual: No hay historial de qué validaciones se aplicaron.

Posible Mejora:

```
public class ReporteValidaciones
{
    private List<RegistroValidacion> validacionesAplicadas =
        new List<RegistroValidacion>();

    public class RegistroValidacion
    {
        public DateTime Timestamp { get; set; }
        public string TipoValidacion { get; set; }
        public string Rango { get; set; }
        public string Pregunta { get; set; }
        public bool Exitosa { get; set; }
    }

    public void ExportarReporte(string rutaArchivo)
    {
        // Genera archivo CSV o Excel con historial
    }
}
```

Patrones de Código y Mejores Prácticas

1. Patrón MVC Adaptado

Estructura Actual:

- **Model:** `_libroCenso`, `_rangoCapturado` (datos en memoria)
- **View:** Controles de Windows Forms
- **Controller:** Métodos de eventos (`btnAplicar_Click`, etc.)

Mejora Propuesta:

```
// Separar lógica en una clase ServicioValidaciones
public class ServicioValidaciones
{
    private Excel.Workbook libro;

    public ServicioValidaciones(Excel.Workbook libroParam)
    {
        libro = libroParam;
    }

    public void AplicarValidacionDecimales(Excel.Range rango)
    {
        // Lógica aquí
    }

    public void AplicarValidacionCatalogos(Excel.Range rango, Excel.Range opciones)
    {
        // Lógica aquí
    }
}

// En formulario:
private ServicioValidaciones servicio;

private void Form_Load(object sender, EventArgs e)
{
    servicio = new ServicioValidaciones(_libroCenso);
}

private void btnAplicar_Click(object sender, EventArgs e)
{
    if (chkDecimales.Checked)
    {
        servicio.AplicarValidacionDecimales(_rangoCapturado);
    }
}
```

2. Manejo de Errores Consistente

Patrón Recomendado:

```
private void OperacionCritica()
{
    try
    {
        // Validaciones previas
        if (_rangoCapturado == null)
            throw new InvalidOperationException("Rango no capturado");

        // Operación
        _rangoCapturado.FormatConditions.Add(...);
    }
    catch (InvalidOperationException ex)
    {
        MessageBox.Show(ex.Message, "Validación",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
    catch (System.Runtime.InteropServices.COMException ex)
    {
        MessageBox.Show($"Error al acceder a Excel: {ex.Message}", "Error COM",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error inesperado: {ex.Message}", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
        System.Diagnostics.Debug.WriteLine(ex.StackTrace);
    }
}
```

3. Constantes y Configuración

Actual: Valores hardcoded en métodos.

Mejora:

```
public static class ConfiguracionValidaciones
{
    // Colores
    public static readonly System.Drawing.Color ColorError =
        System.Drawing.Color.Yellow;
    public static readonly System.Drawing.Color ColorTextoError =
        System.Drawing.Color.Red;

    // Mensajes
    public const string MsgRangoNoCapturado = "Primero captura un rango";
    public const string MsgValidacionExitosa = "Validación aplicada con éxito";
    public const string MsgErrorAplicar = "Error al aplicar validación: ";
}
```

```
// Límites
public const int MaxOpcionesCatalogo = 256;
public const int MaxRangosMomentaneos = 10;
}

// Uso:
formato.Interior.Color =
ColorTranslator.ToOle(ConfiguracionValidaciones.ColorError);
MessageBox.Show(ConfiguracionValidaciones.MsgValidacionExitosa);
```

4. Logging y Debugging

Implementación Básica:

```
public static class Logger
{
    public static void Info(string mensaje)
    {
        System.Diagnostics.Debug.WriteLine($"[INFO] {DateTime.Now:HH:mm:ss} - {mensaje}");
    }

    public static void Error(string mensaje, Exception ex = null)
    {
        System.Diagnostics.Debug.WriteLine(
            $"[ERROR] {DateTime.Now:HH:mm:ss} - {mensaje}");
        if (ex != null)
            System.Diagnostics.Debug.WriteLine(ex.StackTrace);
    }
}

// Uso:
Logger.Info($"Capturado rango: {_rangoCapturado.Address}");
Logger.Error("Error al aplicar formato", ex);
```

Troubleshooting y Casos Especiales

Problema 1: "Operación no válida para el tipo de objeto"

Síntoma: Excepción al intentar aplicar FormatCondition a rango no válido.

Causa Posible: Rango contiene objetos que no permiten formato (imágenes, gráficos).

Solución:

```
if (seleccion is Excel.Range)
{
    Excel.Range rango = (Excel.Range)seleccion;

    // Validar que es rango puro (sin imágenes)
    bool esRangoPuro = !rango.Address.Contains("!");

    if (esRangoPuro)
    {
        // Proceder con validación
    }
}
```

Problema 2: Validación de Catálogos no muestra opciones

Síntoma: Lista desplegable no tiene valores visibles.

Causa Posible: Referencia a rango en hoja diferente sin nombre completo.

Solución:

```
// INCORRECTO
string formula = "=A1:A10"; // Assume misma hoja

// CORRECTO
string formula = "='Hoja de Opciones'!$A$1:$A$10"; // Nombre explícito
// O mejor aún:
string formula = "=" + rangoOpciones.get_Address(true, true,
    Excel.XlReferenceStyle.xlA1, true); // Incluye nombre
```

Problema 3: Fórmulas de validación no funcionan en otros idiomas

Síntoma: Fórmula con ESNUMERO falla en Excel en inglés.

Causa: Función ESNUMERO es específica de español (ISNUMBER en inglés).

Solución:

```
// Usar funciones que funcionan en todos los idiomas
// EVITAR: ESNUMERO, CONTAR.SI
// USAR: Formatos que se traducen automáticamente

// En lugar de:
string formula = $"=Y(ESNUMERO({direccion}), TRUNCAR({direccion})<>{direccion})";
```

```
// Usar FormulaLocal y dejar que Excel traduzca:
celda.FormulaLocal = $"=Y(ESNUMERO({direccion}), TRUNCAR({direccion})<>
{direccion})";
```

Problema 4: Rango no contiguo no se valida correctamente

Síntoma: Validación solo se aplica al primer área del rango.

Causa: Obtención de dirección solo considera primer celda.

Solución:

```
// Procesar cada área por separado
for (int i = 1; i <= _rangoCapturado.Areas.Count; i++)
{
    Excel.Range area = (Excel.Range)_rangoCapturado.Areas[i];

    // Aplicar validación a cada área
    area.FormatConditions.Add(...);
}
```

Problema 5: "El objeto ha sido eliminado"

Síntoma: Excepción al acceder a _rangoCapturado después de cierto tiempo.

Causa: Workbook fue cerrado o rango fue deletionado.

Solución:

```
private bool EsRangoValido(Excel.Range rango)
{
    try
    {
        // Intentar acceder a propiedad básica
        _ = rango.Address;
        return true;
    }
    catch (System.Runtime.InteropServices.COMException)
    {
        return false; // Rango no es válido
    }
}

// Uso:
if (!EsRangoValido(_rangoCapturado))
{
```

```
    MessageBox.Show("El rango se ha eliminado o el archivo se cerró.",  
        "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Warning);  
    _rangoCapturado = null;  
}
```

Problema 6: InputBox devuelve string en lugar de Range

Síntoma: Cast a Excel.Range falla, devuelve string.

Causa: Tipo 8 en InputBox no funcionó correctamente.

Solución:

```
object resultado = excelApp.InputBox(  
    "Mensaje...",  
    "Título...",  
    Type.Missing, Type.Missing, Type.Missing, Type.Missing, Type.Missing, 8);  
  
// Validar tipo de retorno  
if (resultado is Excel.Range)  
{  
    Excel.Range rango = (Excel.Range)resultado;  
    // Proceder  
}  
else if (resultado is string)  
{  
    // El usuario escribió texto en lugar de seleccionar  
    MessageBox.Show("Por favor, selecciona con el ratón, no escribas.",  
        "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);  
}  
else if (resultado is bool && (bool)resultado == false)  
{  
    // Usuario canceló  
    MessageBox.Show("Operación cancelada", "Aviso");  
}
```

Casos de Uso Avanzados

Caso 1: Validación Encadenada

Aplicar múltiples validaciones en secuencia:

```
private void AplicarValidacionesEnCadena()  
{  
    try
```

```
{
    // Primero: Validar que son números
    chkDecimales.Checked = true;
    btnAplicar_Click(null, null);

    // Luego: Validar rango mínimo-máximo
    chkRangoNumerico.Checked = true;
    btnAplicar_Click(null, null);

    // Finalmente: Mostrar resumen
    MessageBox.Show("Todas las validaciones se aplicaron correctamente",
        "Resumen", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
catch (Exception ex)
{
    MessageBox.Show($"Error en validación encadenada: {ex.Message}");
}
}
```

Caso 2: Validación Condicional Basada en Otra Columna

Validación que depende del valor en otra columna:

```
private void AplicarValidacionCondicional(Excel.Range rango,
    Excel.Range rangoCondicion)
{
    try
    {
        // Fórmula: Si columna A = "X", entonces requerir valor en B
        string formulaCondicional =
            $"=SI(A1=\"X\", B1<>\"\", VERDADERO)";

        rango.Validation.Add(
            Excel.XlDvType.xlValidateCustom,
            Excel.XlDvAlertStyle.xlValidAlertStop,
            Excel.XlFormatConditionOperator.xlBetween,
            formulaCondicional,
            Type.Missing);
    }
    catch (Exception ex)
    {
        Logger.Error("Error en validación condicional", ex);
        throw;
    }
}
```

Caso 3: Auditoría de Cambios

Registrar qué validaciones se aplicaron y cuándo:

```
public class AuditoriaValidaciones
{
    private List<EventoValidacion> eventos = new List<EventoValidacion>();

    public class EventoValidacion
    {
        public DateTime Fecha { get; set; }
        public string TipoValidacion { get; set; }
        public string RangoAfectado { get; set; }
        public string UsuarioSistema { get; set; }
        public string NombrePregunta { get; set; }
        public bool Exitosa { get; set; }
        public string DetalleError { get; set; }
    }

    public void RegistrarValidacion(EventoValidacion evento)
    {
        eventos.Add(evento);
        Logger.Info(
            $"Validación {evento.TipoValidacion} en {evento.RangoAfectado} - " +
            (evento.Exitosa ? "EXITOSA" : $"ERROR: {evento.DetalleError}"));
    }

    public void ExportarAuditoria(string rutaArchivoExcel)
    {
        // Crear libro con información de auditoría
    }
}
```

Conclusión

SAVCNG ExcelDNA es un complemento robusto y extensible para validación de datos en Excel. Su arquitectura modular permite fácil mantenimiento y expansión de funcionalidades.

Fortalezas: ✓ Interfaz intuitiva y accesible ✓ Soporte multiidioma mediante funciones localizadas ✓ Validaciones versátiles (decimales, catálogos, NS) ✓ Detección automática de contexto (número de pregunta) ✓ Alertas automáticas para respuestas NS

Áreas de Mejora Identificadas:

- Implementar patrón Service para separar lógica
- Agregar sistema de historial/auditoría
- Mejorar caché de rangos de catálogos
- Internacionalización de interfaz de usuario

- Soporte para validaciones adicionales (fecha, regex)

Versión: 2.0.0.0

Última Actualización: 9 de enero de 2025

Estado: En Desarrollo Continuo